



PROMPT ENGINEERING AND ARTIFICIAL INTELLIGENCE · 2º SEMESTRE

Embeddings e busca semântica com ChromaDB

Aula 05/14 — Módulo 2: RAG

 Prof. Jorge Luiz Gomes

 1h40min

 nomic-embed-text · ChromaDB

 Andaime 45%

Agenda da Aula

- 1 O que é um embedding — a intuição geométrica do significado 20 min
- 2 Similaridade cosseno — como o sistema encontra documentos relevantes 15 min
- 3 nomic-embed-text via Ollama Cloud — setup e OllamaEmbeddings no LangChain 10 min
- 4 ChromaDB — add(), query(), persist() e metadata filtering 15 min
- 5 🚀 Demo — indexar 20 frases e ver busca semântica vs. keyword 15 min
- 6 🖥️ Lab — coleção ChromaDB com textos do domínio (45% lacunas) 25 min

🎯 OBJETIVO DA AULA

Entender como texto vira número e como números permitem encontrar documentos por *significado* — não por palavras-chave. Ao final, o grupo tem uma coleção ChromaDB do domínio pronta para o pipeline RAG da Aula 06.

Glossário

Embedding /ẽ.bé.dín/

Vetor de números reais que representa o significado de um texto — palavras e frases com sentidos parecidos ficam próximas nesse espaço.

Analogia: como coordenadas GPS, mas em vez de latitude/longitude, cada eixo representa uma nuance de significado.

Vetor

Lista ordenada de números. O nomic-embed-text gera vetores de 768 dimensões — 768 números que, juntos, codificam o significado do texto.

Similaridade cosseno

Métrica que mede o ângulo entre dois vetores. Vai de -1 (opostos) a 1 (idênticos) — na prática, de 0 a 1 para textos.

Analogia: dois ponteiros de relógio apontando quase para o mesmo lugar têm ângulo pequeno = alta similaridade.

ChromaDB

Vector store (banco de dados de vetores) local e gratuito. Armazena embeddings, busca por similaridade e persiste no disco.

Coleção (collection)

Conjunto de documentos dentro do ChromaDB — cada um com texto, embedding, id e metadados opcionais. Equivalente a uma tabela num banco relacional.

Retriever

Componente do LangChain que recebe uma pergunta, converte em embedding e devolve os documentos mais similares — a peça que conecta o ChromaDB à chain RAG da Aula 06.

nomi-embed-text

Modelo de embedding de 274M de parâmetros usado no semestre, gratuito via Ollama Cloud. Gera vetores de 768 dimensões com janela de 8.192 tokens.

Persistência (persist directory)

Pasta onde o ChromaDB salva a coleção em disco, permitindo recarregar os embeddings sem recalculá-los numa nova sessão.

01

O que é um embedding

Texto → vetor de números que codifica significado. Palavras com sentidos parecidos ficam próximas no espaço vetorial — independente das palavras usadas.

Embedding — transformar significado em coordenadas

Um embedding é um vetor de números reais onde **proximidade geométrica = similaridade de significado**. O modelo aprendeu essa geometria durante o treinamento:

REPRESENTAÇÃO SIMPLIFICADA — 4 DIMENSÕES (NA PRÁTICA: 768)

"pizza"	0.8	0.3	0.1	0.7	→ [0.8, 0.3, 0.1, 0.7, ...]
"macarrão"	0.7	0.3	0.2	0.6	→ [0.7, 0.3, 0.2, 0.6, ...] ← próximo de pizza
"carro"	0.1	0.8	0.9	0.1	→ [0.1, 0.8, 0.9, 0.1, ...] ← longe de pizza e macarrão

✅ O QUE EMBEDDING CAPTURA

Semântica, contexto, relações de significado. "Pizza" e "lasanha" têm alta similaridade porque aparecem em contextos parecidos nos dados de treinamento — não porque compartilham letras.

⚡ POR QUE É PODEROSO PARA BUSCA

"Qual é o custo do plano?" pode encontrar "O valor mensal é R\$ 49,90" — sem uma palavra em comum. A busca por keyword tradicional falharia; embeddings encontram por significado.

Similaridade cosseno — a métrica de proximidade

O sistema mede o **ângulo entre vetores** para determinar similaridade. Quanto menor o ângulo, mais similares os significados:

DEMO — QUERY: "COMIDA ITALIANA" (SEM USAR ESSA FRASE NO CORPUS)

🍕 "A pizza napolitana usa tomate San Marzano"		0.94
🍝 "Macarrão al dente com molho bolonhesa"		0.91
🍝 "Lasanha com ricota e queijo parmesão"		0.89
🌮 "Tacos com carne temperada e pico de gallo"		0.62
🚗 "O motor do carro faz barulho estranho"		0.12

💡 SIMILARIDADE COSSENO — A FÓRMULA INTUITIVA

$\cos(\theta) = (A \cdot B) / (|A| \times |B|)$ · Vai de -1 (opostos) a 1 (idênticos). Para textos, raramente fica negativo — a faixa prática é 0 (sem relação) a 1 (idêntico). Um threshold de 0.75+ costuma indicar relevância real.

02

nomic-embed-text via Ollama Cloud

274M parâmetros · janela de 8.192 tokens · gratuito via Ollama Cloud · o modelo de embedding padrão do semestre — sem HuggingFace, sem chave de API extra.

nomic-embed-text — setup no LangChain e Ollama

O mesmo cliente Ollama que você usa para o LLM serve para embeddings. Só troca o modelo:

```
!pip install langchain-ollama langchain-community chromadb -q

from langchain_ollama import OllamaEmbeddings
import os
from google.colab import userdata

os.environ["OLLAMA_HOST"] = "https://ollama.com"
os.environ["OLLAMA_API_KEY"] = userdata.get("OLLAMA_API_KEY")

# Modelo de embedding — não é um LLM de geração de texto
embeddings = OllamaEmbeddings(
    model="nomic-embed-text", # 274M params, janela 8k tokens
)

# Gerar embedding de uma frase — retorna lista de floats
vetor = embeddings.embed_query("Como fazer feijoada?")
print(f"Dimensões: {len(vetor)}") # → 768 floats
print(f"Primeiros 5: {vetor[:5]}") # → [-0.024, 0.031, ...]
print(f"Tipo: {type(vetor[0])}") # → <class 'float'>

# Embeddings de múltiplos documentos de uma vez
docs = ["Pizza com queijo", "Macarrão ao sugo", "Motor de carro"]
vetores = embeddings.embed_documents(docs)
print(f"Documentos: {len(vetores)}") # → 3
print(f"Dims cada: {len(vetores[0])}") # → 768
```

✅ EMBED_QUERY VS. EMBED_DOCUMENTS

`embed_query("texto")` → um vetor (para a pergunta). `embed_documents(["a", "b"])` → lista de vetores (para os documentos). O nomic-embed-text aplica prefixos diferentes internamente: "search_query:" para queries e "search_document:" para documentos — essa assimetria melhora a qualidade da busca.

Calcular similaridade cosseno — o mecanismo por baixo

Antes de usar o ChromaDB, entender o cálculo manual ajuda a intuir o que acontece por baixo:

SIMILARIDADE_MANUAL.PY

```
import numpy as np

def similaridade_cosseno(v1: list, v2: list) -> float:
    """Calcula similaridade entre dois vetores de embedding."""
    a, b = np.array(v1), np.array(v2)
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))

# Comparar pares de frases
frases = {
    "pizza": "Pizza napolitana com mozzarella",
    "lasanha": "Lasanha com queijo e presunto",
    "futebol": "Futebol é o esporte mais popular",
    "carro": "Motor do carro faz barulho",
}

vetores = {k: embeddings.embed_query(v) for k, v in frases.items()}

query_vec = embeddings.embed_query("comida italiana")

for nome, vec in vetores.items():
    sim = similaridade_cosseno(query_vec, vec)
    print(f"{nome:10s}: {sim:.4f}")

# pizza:    0.8945 ← alta similaridade
# lasanha:  0.8712 ← alta similaridade
# futebol:  0.4321 ← baixa similaridade
# carro:    0.2104 ← muito baixa similaridade
```

03

ChromaDB

Vector store local e gratuito — armazena embeddings, faz busca por similaridade e persiste no disco. A peça de armazenamento do pipeline RAG.

ChromaDB — add, query e persist

ChromaDB organiza documentos em **coleções**. Cada documento tem: texto, embedding, id e metadados opcionais:

●●●CHROMADB_BÁSICO.PY

```
import chromadb
from chromadb import Settings

# Cliente persistente – salva no disco (não perde ao reiniciar)
client = chromadb.PersistentClient(path="/content/chroma_db")

# Criar ou recuperar uma coleção
colecao = client.get_or_create_collection(
    name="dominio_grupo",
    metadata={"hnsw:space": "cosine"}, # usar similaridade cosseno
)

# Adicionar documentos com embeddings pré-computados
textos = ["Pizza napolitana com mozzarella", "Lasanha bolonhesa com ricota"]
vetores = embeddings.embed_documents(textos)

colecao.add(
    documents=textos,
    embeddings=vetores,
    ids=["doc_1", "doc_2"],
    metadatas=[{"categoria": "italiana"}, {"categoria": "italiana"}],
)

# Buscar os k documentos mais similares à query
query_vec = embeddings.embed_query("comida italiana")
resultados = colecao.query(
    query_embeddings=query_vec,
    n_results=2,
)

print(resultados["documents"]) # → ["Pizza napolitana...", "Lasanha..."]
print(resultados["distances"]) # → [[0.10, 0.13]] (distância, não similaridade)
```

⚠️ DISTANCE VS. SIMILARITY

O ChromaDB retorna **distância**, não similaridade. Com espaço cosseno: distância = 1 – similaridade. Distância 0.10 = similaridade 0.90. Quanto *menor* a distância, *mais* similar.

📖 FUNDAMENTAÇÃO — EPHEMERAL VS. PERSISTENT, E POR QUE GERAR O EMBEDDING FORA DO CHROMA

Polzer (*RAG with Python Cookbook*, O'Reilly, 2026) descreve dois modos de cliente Chroma: **ephemeral**, que guarda tudo em memória e perde os dados ao encerrar o processo, e **persistent** — o usado nesta aula —, que grava em disco e sobrevive a um restart. O autor recomenda algo que o código acima já faz: gerar o embedding no seu próprio código (`embed_documents`) e usar o Chroma só para guardar e buscar, em vez de depender da geração automática embutida. Isso é conveniente para prototipagem, mas prende seu projeto ao Chroma; gerando o vetor por fora, trocar para PostgreSQL/pgvector ou Pinecone mais tarde é só trocar o destino do `.add()`.

Metadata filtering — filtrar antes de buscar

Metadados permitem pré-filtrar a coleção antes da busca por similaridade — indispensável em RAG sobre múltiplas fontes ou categorias:

```
METADATA_FILTERING.PY

# Adicionar documentos com metadados ricos
colecao.add(
    documents=[
        "Pizza margherita é originária de Nápoles",
        "Frango à parmegiana – clássico brasileiro",
        "Churrasco gaúcho com costela e linguiça",
        "Feijoada completa com laranja",
    ],
    embeddings=embeddings.embed_documents([...]),
    ids=["d1", "d2", "d3", "d4"],
    metadatas=[
        {"culinaria": "italiana", "pais": "italia"},
        {"culinaria": "brasileira", "pais": "brasil"},
        {"culinaria": "brasileira", "pais": "brasil"},
        {"culinaria": "brasileira", "pais": "brasil"},
    ],
)

# Buscar SOMENTE culinária brasileira
res = colecao.query(
    query_embeddings=[embeddings.embed_query("prato com carne")],
    n_results=2,
    where={"culinaria": "brasileira"}, # filtro antes da busca
)

# → pizza excluída mesmo que tenha boa similaridade com "carne"
# → retorna somente churrasco e feijoada

# Filtro com operador AND
colecao.query(..., where={"$and": [{"pais": "brasil"}, {"culinaria": "brasileira"}]})
```

💡 METADATA FILTERING É O QUE TORNA RAG ESCALÁVEL

Sem filtro, buscar em 100.000 documentos compara todos os vetores. Com filtro de metadata (ex: apenas documentos do semestre atual, apenas categoria X), o ChromaDB pré-filtra antes de calcular similaridade — muito mais eficiente.

ChromaDB via LangChain — a integração declarativa

O LangChain tem um wrapper sobre o ChromaDB que usa a mesma API LCEL e funciona diretamente como **retriever** no pipeline RAG:

```
from langchain_community.vectorstores import Chroma
from langchain_ollama import OllamaEmbeddings
from langchain_core.documents import Document

embeddings = OllamaEmbeddings(model="nomic-embed-text")

# Criar vector store com LangChain Documents
documentos = [
    Document(page_content="Pizza napolitana", metadata={"fonte": "livro_1"}),
    Document(page_content="Macarrão ao sugo", metadata={"fonte": "livro_1"}),
    Document(page_content="Churrasco gaúcho", metadata={"fonte": "livro_2"}),
]

# from_documents calcula embeddings e salva tudo automaticamente
db = Chroma.from_documents(
    documents=documentos,
    embedding=embeddings,
    persist_directory="/content/chroma_langchain",
)

# Busca direta por similaridade
docs = db.similarity_search("comida italiana", k=2)
for d in docs:
    print(d.page_content, , d.metadata)

# Converter em retriever para uso em chain (Aula 06)
retriever = db.as_retriever(
    search_type="similarity",
    search_kwargs={"k": 3},          # recuperar top-3
)

# → este retriever vai direto na chain RAG da Aula 06
```

CHROMA_LANGCHAIN.PY

05

Lab — Notebook Aluno

45% de lacunas — criar coleção ChromaDB com textos do domínio do grupo, testar 5 queries semânticas e documentar a qualidade dos resultados.

Notebook Aluno — 45% de lacunas

Construir a coleção ChromaDB do domínio. Ela será a base do CKP02 (RAG completo, Aula 07).

AULA_05_2SEM_ALUNO.IPYNB

```
!pip install langchain-ollama langchain-community chromadb numpy -q

from langchain_ollama import OllamaEmbeddings
from langchain_community.vectorstores import Chroma
from langchain_core.documents import Document
import os
from google.colab import userdata

os.environ["OLLAMA_HOST"] = "https://ollama.com"
os.environ["OLLAMA_API_KEY"] = userdata.get("OLLAMA_API_KEY")

# 🚩 LACUNA 1: instancie OllamaEmbeddings com o modelo correto
embeddings = OllamaEmbeddings(model=[])

# 🚩 LACUNA 2: crie 10+ documentos do domínio do grupo
# Use Document(page_content="...", metadata={"categoria": "..."})
documentos = [
    Document(page_content=[], metadata={"categoria": []}),
    # ... mais 9+ documentos
]

# 🚩 LACUNA 3: crie o vector store com from_documents
db = Chroma.from_documents(
    documents=[],
    embedding=[],
    persist_directory="/content/chroma_ckp02",
)

# 🚩 LACUNA 4: faça 5 queries semânticas e documente a qualidade
queries = [[], [], [], [], []]
for q in queries:
    res = db.similarity_search_with_score(q, k=3)
    print(f"\nQuery: {q}")
    for doc, score in res:
        print(f" [{score:.3f}] {doc.page_content}")

# Converter em retriever para a Aula 06
retriever = db.as_retriever(search_kwargs={"k":3})
print(f"\nRetriever pronto para Aula 06: {type(retriever)}")
```

Persistência e recarga da coleção

Embeddings são computacionalmente caros — você não quer recalculá-los a cada sessão. O ChromaDB persiste no disco e recarrega instantaneamente:

```
PERSISTENCIA_RELOAD.PY

# — SESSÃO 1: criar e salvar —
db = Chroma.from_documents(
    documents=documentos,
    embedding=embeddings,
    persist_directory="/content/chroma_ckp02", # salva automaticamente
)
print(f"Documentos salvos: {db._collection.count()}")

# — SESSÃO 2: recarregar sem recalculá-los —
db_recarregado = Chroma(
    persist_directory="/content/chroma_ckp02", # lê do disco
    embedding_function=embeddings,           # para gerar embeddings de queries
)
print(f"Documentos recuperados: {db_recarregado._collection.count()}")

# Adicionar documentos a uma coleção existente
novos_docs = [Document(page_content="Novo documento do domínio")]
db_recarregado.add_documents(novos_docs)
# Só os novos docs têm embeddings computados — os antigos já estão no disco

# No Google Colab: copiar a pasta para o Drive para persistência entre sessões
!cp -r /content/chroma_ckp02 /content/drive/MyDrive/chroma_ckp02
```

⚠️ GOOGLE COLAB NÃO PERSISTE ENTRE SESSÕES

O `/content/` do Colab é temporário — se a sessão reiniciar, a pasta some. Para o CKP02, monte o Google Drive e salve a coleção lá, ou reindexe os documentos sempre que precisar.

similarity_search_with_score — avaliar a qualidade da recuperação

Para o CKP02, você vai precisar saber se os documentos recuperados são de fato relevantes. `similarity_search_with_score` retorna os scores junto com os documentos:

```
BUSCA_COM_SCORE.PY

# Busca com scores (distância cosseno – menor = mais similar)
resultados = db.similarity_search_with_score(
    "comida italiana",
    k=5,
)

for doc, score in resultados:
    # score é distância (0 = idêntico, 1 = sem relação)
    similaridade = 1 - score # converter para similaridade
    relevante = "✅" if similaridade > 0.75 else "⚠️"
    print(f"{relevante} [{similaridade:.3f}] {doc.page_content[:60]}")

# Exemplo de saída:
# ✅ [0.943] Pizza margherita com tomate fresco e mozzarella
# ✅ [0.921] Lasanha bolonhesa com ricota cremosa
# ✅ [0.897] Macarrão al dente com pesto de manjericão
# ⚠️ [0.623] Frango à parmegiana – clássico brasileiro
# ⚠️ [0.412] Tacos com carne temperada

# Filtrar por threshold de relevância
THRESHOLD = 0.75
relevantes = [doc for doc, score in resultados if (1-score) > THRESHOLD]
```

💡 THRESHOLD 0.75 — UMA HEURÍSTICA PRÁTICA

Não existe threshold universal — depende do domínio e do modelo. Para o nomic-embed-text, 0.75+ costuma indicar relevância real. O RAGAS da Aula 07 vai medir isso de forma mais rigorosa com métricas de faithfulness e answer relevancy.

Embeddings e busca semântica no mercado

STREAMING E RECOMENDAÇÃO

Recomendação de músicas e vídeos

Cada faixa ou vídeo vira um vetor baseado em características acústicas, visuais ou de texto. "Itens parecidos" = vizinhos no espaço vetorial. O mesmo princípio do ChromaDB, em escala de dezenas de milhões de itens.

BUSCA JURÍDICA

Busca de jurisprudência

Plataformas de busca em bases de processos judiciais usam embeddings para achar decisões por significado. "Indenização por atraso de voo" encontra decisões que falam em "dano material por cancelamento de aeronave" — sem palavra em comum.

ORGANIZAÇÃO PESSOAL

Busca em notas e documentos pessoais

Ferramentas de anotação indexam cada nota como embedding. "Ideias de projetos de IA" encontra uma nota chamada "Brainstorm de automação com LLMs" — busca semântica sobre o próprio conteúdo do usuário.

ONDE ISSO APARECE NO CKP02

No CKP02, os PDFs do domínio do grupo serão indexados como embeddings no ChromaDB. Uma pergunta do usuário vira um vetor, os trechos mais similares são recuperados e injetados no contexto do LLM — isso é o RAG que você vai construir nas Aulas 06 e 07.

Erros comuns com embeddings e ChromaDB

ERRO	CAUSA	SOLUÇÃO
<code>ConnectionError</code> ao gerar embedding	<code>os.environ["OLLAMA_HOST"]</code> não definido antes de instanciar <code>OllamaEmbeddings</code>	Definir as variáveis de ambiente antes de qualquer instanciação LangChain
<code>UniqueConstraintError</code> no <code>add()</code>	ID duplicado — tentou adicionar doc com mesmo id já existente na coleção	Usar <code>get_or_create_collection</code> e gerar ids únicos com <code>str(uuid.uuid4())</code>
Resultados irrelevantes na busca	Corpus muito pequeno ou textos genéricos demais para o domínio	Indexar textos mais específicos do domínio; usar textos de 1–3 parágrafos (não frases curtas demais)
Coleção some após reiniciar o Colab	<code>/content/</code> é temporário no Colab	Montar Google Drive e salvar a pasta lá, ou reindexar ao iniciar a sessão
Scores todos iguais a 1.0	Espaço de distância não configurado como cosine — usando distância Euclidiana padrão	Criar coleção com <code>metadata={"hnsw:space": "cosine"}</code>

Síntese da Aula 05



Embedding = significado em coordenadas. Palavras e frases com sentidos parecidos ficam próximas no espaço vetorial — independente das palavras usadas.



Similaridade cosseno mede o ângulo entre vetores. De 0 (sem relação) a 1 (idêntico). O ChromaDB retorna distância — similaridade = $1 - \text{distância}$.



nomic-embed-text — modelo de embedding gratuito via Ollama Cloud. 768 dimensões, janela de 8k tokens. `embed_query` para buscas, `embed_documents` para o corpus.



ChromaDB armazena embeddings, faz busca por similaridade e persiste no disco. `from_documents` → `similarity_search` → `as_retriever()` — prontos para o RAG da Aula 06.



Próximo passo — Aula 06: conectar o retriever a uma chain LCEL. O LLM vai responder sobre documentos reais usando o contexto recuperado pelo ChromaDB. Isso é o pipeline RAG completo.

Perguntas frequentes

PERGUNTA	RESPOSTA
"Por que nomic-embed-text e não o modelo de embedding do OpenAI?"	nomic-embed-text é gratuito via Ollama Cloud e roda sem API key extra. O text-embedding-3 da OpenAI tem custo por uso. Para o semestre, o nomic-embed-text oferece qualidade excelente para a maioria dos domínios — é o padrão aprovado.
"768 dimensões é suficiente ou precisamos de mais?"	Para a maioria das aplicações práticas, 768 dimensões (nomic-embed-text) é mais que suficiente. Modelos maiores usam 1536 ou 3072 dimensões — mais dimensões nem sempre = melhor para domínios específicos. O que importa é a qualidade do treinamento.
"Posso usar o ChromaDB em produção ou é só para desenvolvimento?"	ChromaDB tem modo servidor (não só local) e é usado em produção em aplicações pequenas e médias. Para escala maior, Pinecone, Weaviate e pgvector são opções. Para o semestre e CKP02, ChromaDB local é perfeito.
"O que é FAISS e qual a diferença para o ChromaDB?"	FAISS (Meta) é uma biblioteca de busca vetorial de alto desempenho — muito rápida, mas sem persistência nativa e sem metadados. ChromaDB é um banco de dados completo com persistência, metadados e API HTTP. Para RAG em produção, ambos são usados — ChromaDB para simplicidade, FAISS para velocidade.

Python novo desta aula

CONCEITOS_PYTHON_AULA05_2SEM.PY

```
import numpy as np
import uuid

# 1. numpy – dot product e norma vetorial
a = np.array([0.8, 0.3, 0.5])
b = np.array([0.7, 0.4, 0.6])
np.dot(a, b)          # produto escalar (dot product)
np.linalg.norm(a)     # norma (magnitude) do vetor

# 2. Dict comprehension com zip – gerar vetores por chave
nomes = ["pizza", "carro"]
vetores = [embeddings.embed_query(n) for n in nomes]
mapa = {k: v for k, v in zip(nomes, vetores)} # zip e dict comp

# 3. uuid – gerar IDs únicos para documentos
doc_id = str(uuid.uuid4()) # "f47ac10b-58cc-4372-a567-0e02b2c3d479"

# 4. List comprehension com unpacking de tupla
resultados = [(doc, score)] # retorno do similarity_search_with_score
relevantes = [doc for doc, score in resultados if (1-score) > 0.75]

# 5. f-string com formatação de float
print(f"Score: {score:.3f}") # 3 casas decimais
print(f"Texto: {texto[:60]}") # cortar string em 60 chars
```

Onde estamos no Módulo 2 — RAG



A05 (hoje): Embeddings · Similaridade cosseno · nomic-embed-text · ChromaDB · coleção do domínio pronta



A06: Pipeline RAG completo — load PDF · split · embed · store · retrieve · generate · chain LCEL com retriever



A07: RAG avançado — chunking estratégico · reranking · RAGAS · **entrega CKP02**



A08: Interfaces com Gradio e Streamlit — RAG com URL pública



PARA A AULA 06 — TRAZER OS PDFS

Trazer 2–3 PDFs do domínio do grupo (mínimo 5 páginas cada). Na Aula 06 você vai carregá-los com `PyPDFLoader`, dividir em chunks e indexar no ChromaDB. A coleção de hoje (com textos manuais) vai ser complementada ou substituída pela coleção de PDFs reais.

O que vem na Aula 06 — Pipeline RAG completo

O QUE VOCÊ VAI CONSTRUIR

Um pipeline completo que transforma PDFs em respostas:

PDF → PyPDFLoader → chunks → embed → ChromaDB

Pergunta → embed → query → contexto → LLM → resposta

Tudo numa chain LCEL: retriever | prompt | llm | parser

O QUE A COLEÇÃO DE HOJE JÁ RESOLVE

A coleção ChromaDB construída hoje já tem o `retriever` pronto via `as_retriever()`. Na Aula 06, você só precisa conectar esse retriever à chain LCEL e adicionar o prompt de RAG. O passo pesado (embeddings) já está feito.

 **Ação antes da Aula 06:** salve o notebook de hoje com a coleção pronta. Leve os PDFs do domínio. A Aula 06 vai conectar as peças: retriever + prompt RAG + LLM.

04

Demo ao vivo

Indexar 20 frases sobre temas variados → buscar por "comida italiana" → ver retornar pizza, macarrão e lasanha sem a frase aparecer no corpus.



Demo Prática ao Vivo

O professor indexa ao vivo ~20 frases de temas variados (culinária italiana, culinária brasileira, tecnologia e outros) numa coleção ChromaDB e roda a mesma busca de duas formas: por significado (embedding) e por palavra-chave. A turma compara os resultados lado a lado.

1. INDEXAÇÃO

20 frases viram vetores de 768 dimensões via `nomic-embed-text` e são salvas numa coleção ChromaDB, cada uma com metadado de categoria.

2. BUSCA SEMÂNTICA

Query "**comida italiana**" — que não aparece em nenhuma frase do corpus — retorna pizza, lasanha e macarrão, todas com similaridade acima de **0.89**.

3. BUSCA POR PALAVRA-CHAVE

A mesma pergunta filtrada por substring "italiana" retorna **zero resultados** — a palavra literal não existe no corpus. Essa é a diferença que a aula quer mostrar.



STACK DA DEMO

Google Colab · Ollama Cloud API · `nomic-embed-text` · ChromaDB

Coleção ChromaDB do domínio do grupo ★★

Grupo 3–4 · 25 minutos · Google Colab

1. **Complete as 4 lacunas** — modelo de embedding, 10+ documentos do domínio com metadados, criação do vector store e 5 queries semânticas.
2. **Use metadados significativos** para o domínio — ex: em culinária, uma categoria como "sobremesa"; em direito, a lei aplicável como "CDC". Esses metadados serão usados no RAG avançado da Aula 07.
3. **Documente a qualidade** de cada query num comentário: o resultado faz sentido? A similaridade reflete o esperado?
4. **Desafio:** calcule manualmente a similaridade cosseno entre 2 frases do corpus usando numpy (Slide 08) e compare com o score retornado pelo ChromaDB.

🔗 Gabarito das lacunas

Lacuna 1: o nome do modelo de embedding usado no semestre inteiro — o mesmo configurado no Slide 07.

Lacuna 2: cada documento recebe o texto relevante do domínio do grupo e uma categoria como metadado — repetido para os 10+ documentos da lista.

Lacuna 3: o vector store recebe a lista de documentos e o objeto de embeddings criados nas lacunas anteriores.

Lacuna 4: 5 frases que representam perguntas reais de alguém do domínio — quanto mais distantes das palavras exatas do corpus, melhor o teste de busca semântica.

💡 **Dica de qualidade:** para testar se a busca está boa, use queries que *não* compartilham palavras com os documentos indexados. Se ainda retornar resultados relevantes, o embedding está funcionando bem para o domínio.

Referências

DOCS **ChromaDB** — Documentação oficial: collections, add, query, persist, metadata filtering. docs.trychroma.com

DOCS **LangChain** — OllamaEmbeddings e integração com ChromaDB. python.langchain.com/docs/integrations/vectorstores/chroma

MODELO **Nomic AI** — nomic-embed-text: especificações, benchmarks e casos de uso. huggingface.co/nomic-ai/nomic-embed-text-v1

PAPER **Mikolov, T. et al.** — "Efficient Estimation of Word Representations in Vector Space." (Word2Vec, 2013) — a fundação conceitual dos embeddings modernos. arxiv.org/abs/1301.3781

LIVRO **Goodfellow, I.; Bengio, Y.; Courville, A.** — Deep Learning. Pearson, 2017. Cap. 15: Representação distribuída — a base teórica dos embeddings e espaços vetoriais.

Aula 05 concluída

Texto virou número, número virou busca semântica. Na Aula 06 esse retriever se conecta ao LLM e o chatbot passa a responder sobre documentos reais.

→ [PRÓXIMA AULA — AULA 06 · 14/09](#)

Pipeline RAG completo — load, split, embed, retrieve, generate

Conectar o retriever à chain LCEL. O LLM responde com base nos seus documentos.

 [Trazer para a Aula 06](#)

2–3 PDFs do domínio do grupo (mín. 5 páginas cada). Serão o corpus do CKP02.